

Video Game Programming: Game Architecture

L11

Prof. Fabio Buttussi, PhD
Human-Computer Interaction Laboratory
Department of Mathematics, Computer Science, and Physics
University of Udine - Italy

Architectural approaches

- To make a game, two extreme architectural approaches can be considered:
 - **Monolithic** approach has a main script that controls all game objects
 - **Fully distributed** approach has an independent scrip for each object
- Except for specific cases, both these extreme approaches are unpractical, and an intermediate **modular** approach is preferred
- A typical way to organize a modular game architecture is using a **manager** and possibly **controllers** for each main function

Managers and controllers

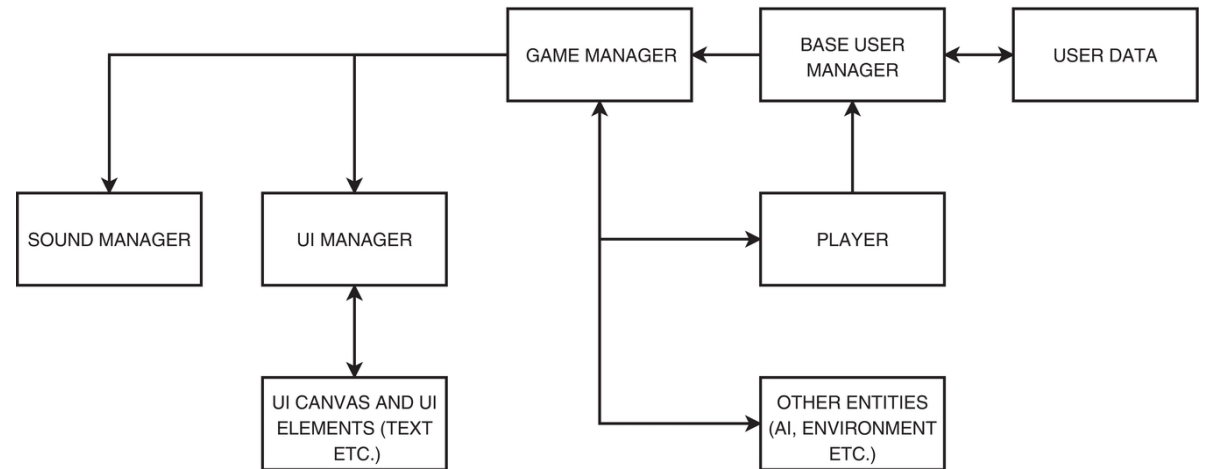
- Murray (C# Game Programming Cookbook for Unity 3D, 2021) makes this distinction:
 - **Managers** deal with overall management
 - **Controllers** deal with systems that managers need to do their jobs
- E.g., a race manager talks to all race controllers attached to players to determine who is winning and race start / end
- For each function, there usually is only one manager in an invisible object and zero to several controllers on game objects

Typical managers and controllers

- Game (logic) manager is the main manager for the game (it can have a specific name such as RaceManager)
- Scene or level manager handles transition between levels
- A simulation manager and AI controllers handle overall simulation aspects and each game object individual AI (e.g., for each NPC)
- Input or player manager deals with players' input
- Log, database, user data and/or analytics manager focus on data
- Audio, network, and UI managers focus on these specific aspects

Example

- An example of game architecture (in fig.) can be found in Ch. 3 of Murray, C# Game Programming Cookbook for Unity 3D, 2nd ed., CRC Press, 2021



Communication

- An important part of a modular architecture is how managers communicate with:
 - the controllers for the function they manage
 - managers that handle other functions
- There are different ways to implement communication:
 - Saving references into variables (assigned by editor or code)
 - Sending messages (in Unity with SendMessage, without return value)
 - Using static variables
 - Using shared data structures

Static variables

- Static variables are accessible to other scripts without a direct reference to a game object
- This is useful when several scripts need to manipulate a variable (e.g., to add points to a score, set the number of player lives)
- Static variables can be public or private:
 - Public static variables exist everywhere and may be accessed by any other script
 - Private static variables are accessible only to any other instances of the same type of class

Shared data structures

- Sometimes, working on single variables can not be sufficient
- When sets of related variables are concerned, more complex data structures are needed
- A well-known example are **blackboards**, i.e., shared memory spaces which various AI components use to store knowledge that can be used by several components and/or multiple times
- Blackboards are typically used to:
 - store expensive checks to avoid the cost of running several times
 - store information used to coordinate AI components between multiple characters or between different components in a character

Singletons

- A singleton is a method to allow access to an **instance** of a class, accessible to all other scripts
- This pattern is ideal for providing globally accessible managers
- For example, given the code in fig., any script can call `AudioManager.instance.objectGroup`;

```
public class AudioManager : MonoBehaviour
{
    public static AudioManager instance;

    public AudioManagerGroup talkGroup;
    public AudioManagerGroup objectGroup;

    ⚙️ Messaggio Unity | 0 riferimenti
    void Awake()
    {
        instance = this;
    }
}
```

Coroutines

- Unity offers coroutine functions that run outside of *Update cycles
- A coroutine is self-contained code that is executed when started with `StartCoroutine(name of the coroutine as a string)`
- Using **yield**, a coroutine can be paused for current update cycle, a set amount of time, or a condition, and then started again
- Coroutines are suited to:
 - time-based actions (e.g., fade, animations)
 - wait on condition actions (e.g., availability of a network resource)

Finite state machines

- Finite state machines are used in games to track game states (e.g., game loaded, level starting)
- They can be implemented in the code as in fig., usually having distinct functions for the different states
- They can also be modeled using game engine tools (e.g., Unity Mecanim)

```
switch(currentGameState)
{
    case GameStates.GameLoaded:
        GameLoaded();
        break;
    case GameStates.GameStarting:
        GameStart();
        break;
}
```

Rule-based systems

- Rule-based systems consists of a set of if-then rules with some checks (if) and actions (then)
- Actions can also need some input to produce some output
- In these systems, there is no explicit state (can be modeled), but all matching rules can be fired using rule matching algorithms as in fig.

```
for each rule that should run at this point in time
  if all checks are satisfied
    if all inputs exist
      consume inputs
      produce outputs
      run success actions
    else
      run failedInputs actions
  else
    run failedChecks actions
```

Pseudo code for rule matching algorithm
from Ch 15 of Rabin, Game AI Pro 360:
Guide to Architecture, CRC Press, 2020

Bibliografy

- Murray, C# Game Programming Cookbook for Unity 3D, 2nd ed., CRC Press, 2021
- Rabin, Game AI Pro 360: Guide to Architecture, CRC Press, 2020 (see also <http://www.gameaipro.com/>)

